Performance Analysis of Apache Spark Job Schedulers for Big Data Processing

Vishnu Prasad Verma
Deptt. of CSE
IIIT Naya Raipur
Raipur, (C.G), India
vishnu@iiitnr.edu.in

Tanisha Sinha
University Teaching Department
CSVTU, Bhilai
Bhilai, (C.G), India
tanisha02sinha@gmail.com

Santosh Kumar
Deptt. of CSE
IIIT Naya Raipur
Raipur, (C.G),India
santosh@iiitnr.edu.in

Nenavath Srinivas Naik
Deptt. of CSE
IIITDM, Kurnool
Kurnool, (AP), India
srinu@iiitk.ac.in

***Abstract*—The rapid increase in data volume due to technological advancements necessitates using robust big data processing frameworks to analyze vast amounts of heterogeneous data. Apache Spark stands out as a state-of-the-art framework for big data processing, known for its fault tolerance, generality, high performance in in-memory data processing, and scalability. One of the key components of Apache Spark is the resilient distributed dataset (RDD) programming paradigm, which aids job scheduling by enabling efficient data partitioning and locality-aware task placement. However, effective job scheduling is crucial for optimizing resource utilization and minimizing job completion times in Apache Spark. This paper performs analysis of various Spark job schedulers, including FIFO (First In First Out), Fair, SJF (Shortest Job First), Adaptive, Service Level Agreement based schedulers, and Bipartite Graph modeling based schedulers. The analysis focuses on metrics that are crucial for performance enhancement, including efficiency, non-violation scenarios, average turnaround time, and average waiting time. By simulating the Apache Spark big data processing cluster and performing extensive experiments, our findings offer essential insights into the strengths and limitations of various schedulers. The research provides guidance for optimizing job scheduling in Apache Spark environments and enhancing future big data processing applications.**

***Index Terms*—Apache Spark, Big data Processing, Distributed Computing Job scheduling, Performance Optimization,**

I. Introduction

As data volumes continue to grow, the need for efficient big data processing and analytical methods becomes ever more essential. Big data processing frameworks application spans various sectors, including banking, business analytics, and customer segmentation. Big data processing frameworks have witnessed a surge in demand due to their inherent flexibility, scalability, and ability to handle rapid data processing tasks. These frameworks are essential for processing large amounts of data and analyzing various factors using powerful big data processing frameworks like Apache Spark [1], Apache Hadoop [2], Flink [4], Presto [5], Kafka [6], and Storm [7]. Consequently, these frameworks are becoming increasingly significant for many futuristic big data processing applications.

Hadoop is mostly focused on batch processing, while Storm is a real-time data stream processing technology. Flink is an adaptable framework with batch and streaming processing capabilities that can manage massive amounts of data. Large datasets from various data sources can be interactively and rapidly queried using the open-source SQL engine Presto. Kafka is a distributed platform that allows pipeline integration at high throughput and low latency, as well as real-time data streaming. However, despite their benefits, these state-of-the-art methods aren't always dependable or effective when managing massive volumes of data. Every framework has its limitations and works effectively in specific circumstances. Compared to the big data processing models discussed above, Apache Spark stands out as a fast and robust large-scale data processing system [8]. Apache Spark is widely used in industry and academia for a range of advanced applications due to its flexible programming model and in-memory processing. While other frameworks excel in particular domains, they fall short of Spark's broad capabilities and overall performance.

Spark with its versatile job scheduling techniques like batch, fair, and FIFO scheduling [9]. Batch processing handles large data volumes in sequence, while fair scheduling evenly distributes resources, and FIFO scheduling processes tasks as they arrive, suitable for small-scale workloads. Spark is highly scalable for workloads of any size thanks to its distributed computing capabilities, which enable it to analyze massive datasets across clusters. Maintaining data integrity in large-scale operations depends on data processing activities' ability to recover smoothly from errors, which is ensured by its fault-tolerant design. Because of this, Spark is the go-to option for complicated data processing jobs in both business and academia, spanning from large-scale scientific simulations to real-time fraud detection.

A typical Spark application consists of one or more jobs, the driver program, and one or more jobs with various stages, each of which is carried out by a group of tasks [10]. The driver program serves as the main control program, executing the user's Spark application and establishing the SparkContext, which facilitates connectivity to the Spark cluster, as shown in Fig. 1. SparkContext acts as the primary interface for interacting with Spark's functionality and execution environment, collaborating with cluster managers to allocate resources and schedule tasks. The Cluster Manager oversees resource allocation throughout the Spark cluster, with examples including YARN [11], Mesos [12], and Spark's standalone manager. Executors, meanwhile, function in worker nodes, executing tasks delegated by the

2024 IEEE Region 10 Symposium (TENSYMP) | 979-8-3503-6486-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/TENSYMP61132.2024.10752267

SparkContext. Each executor can run multiple tasks concurrently and store data either in memory or on disk.

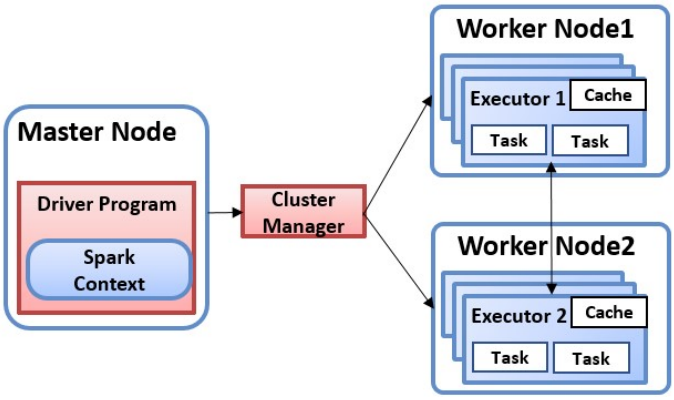

Fig. 1. Apache Spark Cluster Architecture

The Spark cluster architecture as depicted in Figure 1, showcases how Spark leverages a single master process known as the driver for job scheduling across diverse heterogeneous tasks. This driver orchestrates the scheduling process using a hierarchical approach, managing jobs, stages, and tasks. The stages, which represent smaller sets of tasks segmented from independent jobs, mirror the map and reduce phases of a single MapReduce job. Within the Spark framework, two schedulers play key roles:

The DAG Scheduler constructs a Directed Acyclic Graph (DAG) of stages for a job's tasks while actively monitoring materialized RDDs (Resilient Distributed Datasets) and stage outputs. The Task Scheduler, functioning at a low-level or fine-grained level, receives and dispatches assigned jobs or tasks from each phase to various cluster nodes for execution.

In the realm of big data analytics, Spark serves as a cornerstone, offering advanced analytical capabilities crucial for efficient job execution on cloud platforms like Amazon Machine Learning (AML) and Google AI Platform. Enhanced with real-time processing features, Apache Spark complements Hadoop through its components, such as Spark SQL, Spark Streaming, MLlib, GraphX, SparkR, and Spark Core. Its comprehensive suite of tools and features enables seamless integration and analysis of vast datasets across a multitude of industries.

Within the domain of big data processing, job scheduling emerges as a critical component, ensuring the optimization of tasks in both physical and cloud environments. Widely adopted frameworks like Hadoop and Spark are pivotal, with large enterprises and private organizations often managing dedicated clusters for task processing. Additionally, public cloud providers offer deployable infrastructure, facilitating the widespread adoption of big data processing in everyday operations.

II. Scope and Objective

A. The Rise of Big Data and the Spark Revolution

The digital age has ushered in an era of unprecedented data proliferation. Across industries, organizations face massive datasets that encompass structured, semi-structured, and unstructured information—datasets that traditional data processing tools struggle to manage effectively. This phenomenon, commonly known as "big data," presents both challenges and immense opportunities [13]. The true value of big data lies in its potential to reveal hidden patterns, trends, and insights that can inform strategic decision-making, optimize operations, and drive innovation across industries. However, extracting meaningful intelligence from these vast repositories requires robust and scalable processing methods [14]. This is where Apache Spark emerges as a transformative force, offering advanced processing capabilities that enable organizations to harness the power of big data.

B. Spark's Advantage: Flexibility, Scalability, and Speed

Spark's meteoric rise in big data processing stems from its inherent flexibility, scalability, and lightning-fast processing capabilities. Unlike its predecessors, Spark isn't confined to a single processing paradigm. It empowers users with a diverse set of job scheduling techniques, including batch, fair, and FIFO scheduling [15]. Batch processing efficiently handles large data volumes sequentially, while fair scheduling ensures equitable resource allocation across tasks. For smaller-scale workloads, FIFO scheduling prioritizes tasks in the order they are received, maximizing efficiency.

This scheduling versatility complements Spark's seamless integration with leading cloud platforms like Amazon Machine Learning and Google AI Platform [16]. By leveraging distributed processing engines, Spark facilitates efficient job execution and empowers organizations to tap into vast and elastic computing resources, eliminating the limitations of on-premises infrastructure. This underscores Spark's brilliance in big data processing. Figure 2 illustrates the significant mile-

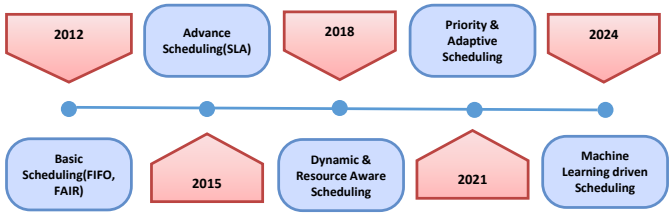

Fig. 2. Milestones in the Evolution of Spark Job Scheduling

stones in the development of Spark job scheduling algorithms. It showcases the remarkable evolution of Apache Spark, a leading big data processing framework. Since its introduction in 2012 as a general-purpose cluster-computing system, Spark has undergone continuous development, transforming into a versatile platform for large-scale data processing. The table I provides a comprehensive comparison of various big data processing frameworks based on different metrics. Each framework, including Spark, Flink, Hadoop, Apache Kafka, Presto,

TABLE I
SUMMARY OF DIFFERENT BIG DATA PROCESSING FRAMEWORKS

Metrics	Spark	Flink	Hadoop	Apache Kafka	Presto	Storm
Usability	Easy-to-use	Easy-to-use	Easy-to-use	Easy-to-use	Easy-to-use	Moderate
Performance	High Efficiency	High Efficiency	Low Efficiency	High Throughput	High Performance	High Throughput
Generality	Yes	Yes	No	No	Yes	No
Flexibility	Yes	Yes	Yes	Yes	Limited	Yes
Scalability	Yes	Yes	Yes	Yes	Yes	Yes
Fault Tolerance	Yes	Yes	Yes	Yes	Yes	Yes
Memory Consumption	Heavy	Heavy	Heavy	Moderate	Moderate	Moderate
Security	Poor	Poor	Strong	Moderate	Poor	Moderate
Learning Curve	Hard-to-learn	Hard-to-learn	Easy-to-learn	Moderate	Easy-to-learn	Moderate
Popularity	Yes	No	No	Yes	Yes	No
Primary Function	Batch/Streaming	Batch/Streaming	Batch	Streaming	SQL querying	Stream processing

and Storm, is evaluated across multiple dimensions such as usability, performance, generality, flexibility, scalability, fault tolerance, memory consumption, security, learning curve, popularity, and primary function.

C. Beyond Speed: Spark's Analytical Powerhouse

Spark's true strength lies not just in its speed but in its comprehensive suite of advanced analytical capabilities. Unlike its predecessor, Hadoop, Spark is built for real-time processing. Components like Spark SQL, Spark Streaming, MLlib, GraphX, SparkR, and Spark Core form a powerful arsenal for tackling diverse analytical tasks.

Spark SQL empowers users to interact with data using familiar SQL syntax, streamlining data querying and manipulation. By utilizing real-time data ingestion and processing, Spark Streaming makes businesses react immediately to ever-evolving situations and make timely decisions. Apache Spark's machine learning library, MLlib, provides an extensive toolset for developing and using complex prediction models. GraphX simplifies the examination of intricate connections found in data, making it ideal for analyzing social networks and fraud detection. SparkR easily interacts with R, a widely used programming language for statistical computing and data visualization. In the end, Spark Core functions as the framework's foundation and offers a general-purpose distributed computing execution engine for jobs [17].

Apache Spark is essential to freely available frameworks designed to address the enormous difficulty of processing big amounts of data. What exactly distinguishes it is its clever application of in-memory data management, an approach that accelerates processing. This masterpiece of architecture is not limited to one language, although it generously embraces the likes of Python, R, Scala, and Java, empowering a broader spectrum of developers to wield its capabilities. Spark possesses a wide range of capabilities, including exceptional machine learning tools and graph processing, as well as the requirements of live streaming. Essential to its structure, that is a lying that is resilient distributed datasets (RDDs) [18], which are essential for preserving data integrity even in the event of a node failure.

However, due to the enormous amount and complexity of data processing tasks, accuracy in scheduling and resource management are equally as important as speed. In light of this, efficient job scheduling is still essential in order to control the unpredictability of execution schedules, improve data affinity, and fully utilize parallelism. To obtain this accurate harmony, a multifaceted strategic approach is necessary through the allocation of dynamic resources [19] to careful task assignment, every modified to maximize the foundational infrastructure's performance. Furthermore, assigning tasks isn't just a matter of random allocation; it meticulously takes into account locality preferences and resource constraints, ensuring that computational resources are utilized with surgical precision.

Spark's reliance on Resilient Distributed Datasets(RDDs) not only makes data transformations seamless but also acts as a safety net against the capriciousness of system errors. RDDs possess the superpower of recomputation, a failsafe mechanism that revives lost data from the ashes of temporary errors, thanks to their ability to track data modifications. In the meantime, a revolution for job scheduling is brought about by adaptive scheduling, a dynamic system that continuously modifies resource allocation in response to needs in real-time. Using more sophisticated strategies to push the limits of machine learning, predictive analytics is utilized to determine the optimal paths to task distribution. [20].

Many challenges arise in large-scale data processing, including organizing massive datasets and deciphering the complex interrelationships between jobs. A further obstacle that must be addressed is data shuffling [21], which calls for creative ways to improve inter-node communication and reduce bottlenecks. Additionally, Spark has to deal with the diversity of workloads, each of which presents a different combination of task complexity and data sizes [22]. Adapting to this kaleidoscope of demands necessitates a nimble-footed approach, one that continuously evolves to meet the ever-shifting sands of computational exigencies.

Fair scheduling [23] ensures equitable resource distribution, suitable for multi-tenant environments but potentially causing resource underutilization. FIFO scheduling [24] is simple and processes jobs in order of submission but lacks prioritization. Dynamic [25] and adaptive scheduling [26] offer improved efficiency and resource allocation but add complexity. Custom schedulers require significant effort to design and implement but can be effective in specialized scenarios.

III. Literature Review

The difficulty of scheduling big data processing jobs has become more and more prominent as big data analytics has grown in popularity. Numerous studies concentrated on maximizing the runtime environment of big data analysis tasks in order to increase the tasks' efficiency of execution. Several of them created models and suggested strategies to improve the performance of big data processing frameworks. Islam *et al.* [27] has the contribution that entails the provision of a model within a cloud environment aimed at training agents using deep reinforcement learning techniques to enhance cost and performance efficiency. This was achieved by emulating a realistic cluster environment and establishing a reward system based on agent actions, whether favorable or unfavorable. Additionally, two deep reinforcement learning-based agents, deep Q-learning neural (DQN) and Reinforcement, were implemented and trained to function as schedulers within the framework. Comparative evaluations were conducted against baseline schedulers to assess the efficacy of the proposed approach.

Yuan *et al.* [28] suggested approach includes a method for multiobjective optimizations specially designed for Distributed Green Data Centers (DGDCs), with the main goal of optimizing revenue for DGDC entities while reducing the likelihood of an average loss. This is accomplished by a cooperative approach to the decision-making process involving allocating tasks among Internet Service Providers (ISPs) and determining task service rates within Green Data Centers (GDCs).

Hu Z *et al.* [29] proposed Directed Acyclic Graph (DAG) scheduling, which is an essential methodology where structures tasks or stages are represented as nodes within a DAG, with edges representing the dependencies between them. Execution is managed by performing tasks and dependencies in parallel within a stage, progressing through stages based on their dependencies to optimize the overall execution process.By following dependency restrictions and ensuring that tasks are completed in the correct order, this approach promotes effective workflow management.

Z. Hu*et al.* [30] proposed SJF scheduling which assigns a higher priority to shorter jobs based on their projected execution time. However, it presents challenges such as increased complexity, lack of predictability, and potential inaccuracies in execution time estimates.

Islam MT *et al.* [31] stated that SLA scheduling organizes and prioritizes tasks based on agreements with clients, such as response time, throughput, and availability. Tasks are scheduled according to customer needs. Challenges include resource overhead, prioritization issues, unpredictability in maintaining SLAs, and fairness in resource allocation based on SLA requirements.

Cheng D *et al.* [32] proposed that adaptive scheduling is a dynamic technique that adjusts task execution and resource allocations based on real-time processing. It leverages system feedback to fine-tune parameters and optimize execution. Limitations include scheduling overhead that may impact performance, as well as challenges in maintaining stability and balance between responsiveness and consistency. Accurate data is crucial for decision-making; poor data quality can lead to scheduling issues.

TABLE II
Evaluation of Scheduling Techniques on Multiple Metrics

Scheduling Approach	Scalability	Efficiency	Fault Tolerance	Adaptability
Shortest Job First (SJF)	Moderate	High	Low	Low
Service Level Agreement (SLA)	High	High	Moderate	High
Adaptive Scheduling	High	High	High	High
Data-driven Priority Scheduling	High	High	Moderate	High
Bipartite Graph	High	High	Moderate	Moderate
FIFO (First-In-First-Out)	High	Low	Low	Low
Fair Scheduling	Moderate	Moderate	Moderate	Moderate

Ajila T *et al.* [33] proposed that data-driven priority scheduling is a job scheduling approach that leverages patterns in task execution and resource allocation, using predictive, historical, and real-time data to optimize performance. This method analyzes data to make informed scheduling decisions that improve efficiency. However, it faces challenges related to complexity, data quality, overhead, adaptability, and maintaining fairness.

Fu Z*et al.* [34] has the contribution that bipartite graph modeling in scheduling represents relationships between tasks and resources to optimize task scheduling and resource allocation. One set of nodes corresponds to tasks, while the other set represents resources such as processing units, with edges indicating task-resource assignments. This approach can be complex due to the need to handle large complexities and identify optimal solutions, as well as real-time adjustments that may introduce overhead. Additionally, managing constraints can be challenging, potentially causing performance issues.

Lu S. *et al.* [35] addresses two limitations hindering Spark's performance: data partitioning and job scheduling in heterogeneous clusters. The authors propose a dynamic partitioning strategy to tackle data skew and a greedy scheduling algorithm that considers both node capabilities and task requirements. Their experiments show these methods improve overall job completion time and resource utilization.

The analysis of various job scheduling approaches has a significant impact on practitioners and researchers in the field of big data processing. The table II compares the Evaluation of Scheduling Techniques on Multiple Metrics for different algorithms from the literature.

IV. Methodology

The methods for simulating and assessing different Spark schedulers are described in this section. This experiment aims to analyze the schedulers according to crucial metrics and parameters that are very crucial in job scheduling in big data processing. The parameters include Efficiency analysis, Non-Violations, Average Waiting Time, and Average Turnaround Time. Its objectives are to evaluate waiting times for job processing, average turnaround times, job completion success rates, and efficiency.

A. Simulation Environment

This article's purpose is to analyze big data processing schedulers like FIFO, Fair, SJF, Adaptive, SLA-based sched-

uler, Priority-based schedulers, bipartite graph modeling, and their approach to managing job execution. The experimentation is done to simulate a big data processing environment in which multiple jobs are submitted to the Spark cluster at the same time and scheduled using various techniques. For our experiment, we developed synthetic data and used the job dataset from previous studies. Each job J_i is characterized by its arrival time t_i, execution time e_i, and resource requirement r_i. The big data processing cluster comprises of N nodes, each with a fixed amount of computational resources.

B. Scheduling Algorithms

1) First In First Out(FIFO): The FIFO scheduling algorithm is the basic and easiest of all the scheduling algorithms available, where jobs are chosen to execute based on their time of arrival. The job that comes gets scheduled first.

$$\text{Priority}(J_i) = 1$$

2) Fair Scheduling: A fair scheduler in Spark allocates resources such that all jobs get, on average, an equal share of computational resources over time. The resources are shared fairly among all the jobs available to schedule at that moment.

$$\text{Share}(J_i) = \frac{1}{\text{Number of Active Jobs}}$$

3) Shortest Job First(SJF): In the SJF scheduler, the jobs that have shorter job completion times are scheduled before longer ones. This scheduler is based on predicting the execution time of jobs scheduled.

$$\text{Priority}(J_i) = \frac{1}{e_i}$$

4) Adaptive Scheduling: Adaptive scheduler in Spark dynamically adjusts resource allocation and execution plans based on runtime information. The adaptive scheduler modifies priorities continuously in response to workload and system state considerations.

$$\text{Priority}(J_i) = \frac{w_1}{e_i} + \frac{w_2}{t_i}$$

where w_1 and w_2 are weights assigned based on the users chosen parameters.

5) Bipartite Graph Modelling Scheduling: The bipartite graph modeling scheduling is based on data locality. A bipartite graph is used to optimize scheduling in Spark by representing the relationship between jobs and computational resources. Bipartite scheduler nodes stand for jobs and resources, whereas edges show possible assignments.

$$\text{Maximize} \sum_{(J_i,R_j)\in E} w_{ij}$$

where w_{ij} is the weight of the edge between job J_i and resource R_j.

6) SLA-based Scheduling: SLA-based scheduler in Spark is designed to optimize job scheduling by prioritizing task and resource allocations to meet predefined performance metrics such as deadline or throughput requirements. In SLA-based scheduler, tasks are sorted based on their Service Level Agreements (SLAs), which specify the required deadline, resource usage limit, or throughput.

$$\text{Priority}(J_i) = \frac{1}{\text{SLA}_i - (t_{\text{current}} - t_i)}$$

7) Priority Scheduling: A priority scheduler in Apache Spark schedules jobs based on assigned priority levels. Jobs with high priority get schedules before low-priority jobs. It takes into account the duration of the job as well as predetermined priority levels by combining SJF with priority scheduling.

$$\text{Priority}(J_i) = \frac{p_i}{e_i}$$

where p_i is the priority of job J_i.

C. Performance Metrics

The scheduling algorithms' efficiency is assessed using the following metrics:

1) Efficiency Analysis (EFF): Efficiency quantifies how well the scheduling algorithm makes use of the resources at its disposal to finish tasks.

$$\text{EFF} = \frac{\text{Total Execution Time}}{\text{Total Resource Time}}$$

where Total Resource Time is the total amount of resources utilized multiplied by the amount of time they are used, and Total Execution Time is the total of all tasks' execution periods.

2) Non-Violation Rate (NVR): The percentage of jobs that satisfy their SLA requirements is measured by NVR.

$$\text{NVR} = \frac{\text{Number of Jobs Meeting SLA}}{\text{Total Number of Jobs}}$$

3) Average Turnaround Time (TAT): Turnaround time is the amount of time needed to finish a work from the moment it is delivered. Better performance is shown by a reduced average turnaround time.

$$\text{TAT}(J_i) = t_{\text{completion}}(J_i) - t_i$$

$$\text{Average TAT} = \frac{\sum_{i=1}^{M} \text{TAT}(J_i)}{M}$$

where M is the total number of jobs.

4) Average Waiting Time (AWT): The amount of time a work waits in the queue before being completed is known as the waiting time. Better performance is shown by a reduced average waiting time.

$$\text{WT}(J_i) = \text{TAT}(J_i) - e_i$$

$$\text{Average WT} = \frac{\sum_{i=1}^{M} \text{WT}(J_i)}{M}$$

The simulation process includes a number of important steps: The cluster configuration, job arrival rates, and resource

requirements are first determined during the initialization phase. After that, a Poisson arrival mechanism is used to build jobs and send them to the cluster. Each scheduling algorithm is then applied to each job in order to assign resources to it. Resource utilization and job execution are simulated during the execution phase. During this phase, several indicators are monitored, such as the average turnaround time, average waiting time, efficiency, and non-violation rate. Finally, an analysis is performed on the collected metrics to determine the effectiveness of each scheduling technique.

V. Results Analysis and Discussion

In this section, we present the experimental setup, result analysis, and discussion on issues and future research directions

A. Experimental setup

We performed extensive simulation experiments on a system with an x64 architecture equipped with an Intel i3 CPU with 2 cores and 4 processors, each operating at 2.00 GHz. We have efficiently made different scheduler programs in python language that act as real schedulers. We developed a Python-based framework that acts as a realistic big data processing cluster test environment, simulating real-world conditions for schedulers. The simulation assesses the effectiveness and dependability of various scheduling algorithms, offering valuable insights for analyzing big data processing. It evaluates the capabilities of schedulers, including FIFO, Fair, SJF, Adaptive, SLA-based scheduler, Priority-based scheduler, bipartite graph modeling, and their approach to managing job deadlines, resource constraints, and overall system efficiency. We have compared the scheduler in the crucial metrics of job scheduling, including efficiency analysis, Non-violation of the deadline, average waiting time, and average turnaround time.

B. Result analysis

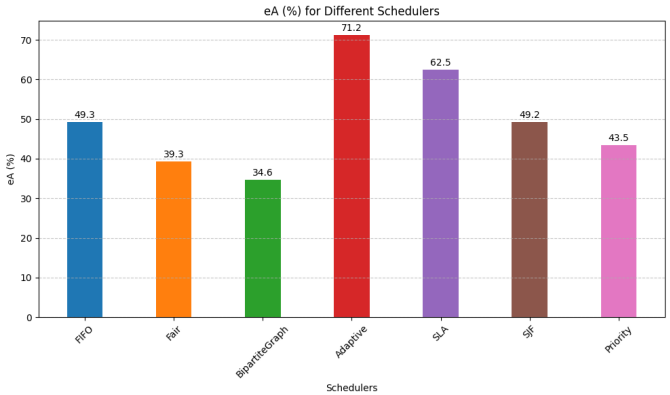

Fig. 3. Efficiency Analysis (eA) for Different Schedulers

We've conducted a comprehensive analysis utilizing a synthetic dataset. The data set consists of three types of jobs, namely compute intensive, I/O intensive, and mixed type. The dataset has information like job requirements which can

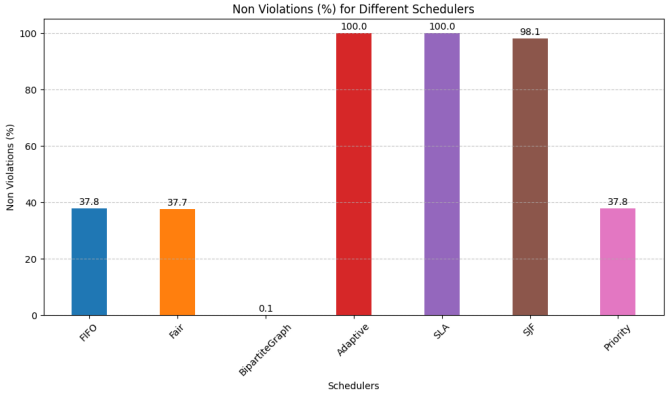

Fig. 4. Non Violations Percentage for Different Schedulers

be found in GitHub link[1]. The dataset encompasses various parameters such as task ID, duration, dependencies, resource requirements, priority, SLA deadline, and arrival time. This dataset serves as the foundation for evaluating the performance of different scheduling algorithms.

Scheduling algorithms were evaluated based on essential metrics that are crucial in gaining high performance, including efficiency analysis (eA) and non-violations percentage, average turnaround time(ATT), and average waiting time (AWT). Efficiency analysis represents efficiency in the utilization of the resource as expected and full completion of tasks within its duration. Larger eA means better efficiency, which means the scheduler is doing an excellent job for the resources it manages and gets tasks completed on time. The non-violations percentage shows the share of jobs that fulfilled all of their SLAs. A higher non-violations percentage means better adherence to SLAs, which is what we are looking for since our end goal is reliability and better quality of service. Lower ATT indicates a more efficient scheduling mechanism, with faster job completion, while lower AWT reflects quicker job scheduling, reducing delays and improving resource utilization. Both metrics are essential for evaluating the effectiveness and responsiveness of Spark scheduling algorithms, especially under varying workloads.

The fig. shows 3 efficiency Analysis (%) that the Adaptive scheduler functions more effectively than all of the other Spark job scheduling algorithms, with an efficiency of 71.2%, according to the efficiency study (eA%) of the other algorithms. Additionally, the SLA scheduler performs admirably, with an efficiency of 62.5%. Both FIFO and SJF schedulers have a slightly more than 49% efficiency comparison. The BipartiteGraph scheduler has the lowest efficiency in this examination, at 34.6%, indicating its relative inefficiency. The Fair and Priority schedulers, on the other hand, show reasonable efficiency at 39.3% and 43.5%, respectively.

Fig. 4 shows Non-Violations (%) for different schedulers, illustrating the reliability of various Spark job scheduling algorithms in adhering to deadlines or SLAs. Both the Adaptive

[1]https://github.com/tanisshaaa/tanisshaaa/blob/main/.csv

and SLA schedulers exhibit perfect reliability, achieving 100% non-violation rates. The SJF scheduler also performs well with a non-violation rate of 98.1%. In contrast, FIFO, Fair, and Priority schedulers show moderate reliability, each with approximately 37.8% non-violation rates. The Bipartite Graph scheduler, however, significantly underperforms with a non-violation rate of just 0.1%, indicating its limited effectiveness in meeting job deadlines or SLAs in this evaluation.

Fig. 5 shows the results for Average Turn around time for various schedulers. The most effective scheduling technique among those looked at is SJF. The SJF (Shortest Job First) scheduler demonstrates the best performance with the lowest average turnaround time of 122 seconds, indicating faster job completion. Other schedulers, including FIFO, Fair, Adaptive, Priority, and BipartiteGraph, exhibit similar performance with turnaround times ranging between 143.42 and 148.02 seconds. However, the SLA scheduler shows a significantly higher average turnaround time of 299.54 seconds, suggesting it is less efficient in terms of job completion speed under the tested conditions.

The average waiting time in seconds for the different scheduling strategies is shown in Fig. 6. With the lowest average waiting time among the algorithms, SJF is 115.9 sec and regularly performs better than the others, suggesting effective task queue management. SLA, on the other hand, shows the greatest average waiting time of 155.04 sec, indicating a possible trade-off with other performance indicators. The other scheduler's FIFO, Fair, Adaptive, priority, and bipartite graphs have 140.5 sec, 139.1 sec, 141.54 sec, 138.56 sec, and 137.8 sec average waiting times, respectively.

our comprehensive evaluation of various Spark job scheduling algorithms highlights the distinctive strengths and limitations of each approach under different workload scenarios. The Adaptive scheduler consistently outperforms others, achieving the highest efficiency (71.2%) and perfect adherence to SLAs (100% non-violations). SLA-based scheduling also demonstrates robust performance, though its higher average turnaround time suggests a trade-off between deadline adherence and job completion speed. The SJF scheduler, with the lowest average turnaround time and waiting time, emerges as an effective strategy for minimizing task latency. Conversely, the Bipartite Graph scheduler underperforms in both efficiency and SLA adherence, making it less suitable for high-demand environments. These findings underscore the importance of selecting the appropriate scheduling algorithm based on specific workload characteristics and performance objectives, ensuring optimal resource utilization and system efficiency in large-scale data processing environments.

C. Issues and Future Directions

For big data processing, effective job scheduling is necessary since it utilizes resources in clusters, and that impacts the framework performance and cost. The analysis in this research has various scheduling methods, highlighting their strengths and weaknesses in scalability, efficiency, fault tolerance, and adaptability to dynamic workload changes. With this knowledge, researchers and practitioners can make well-informed judgments about their unique big data necessity.

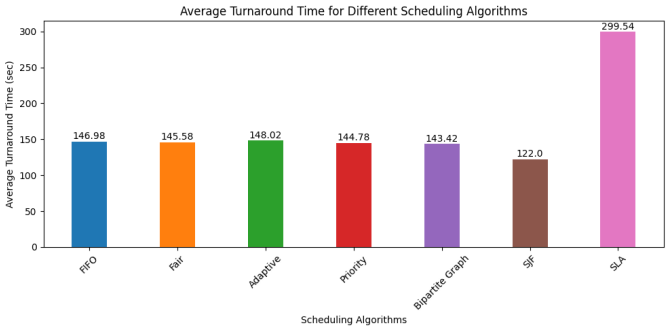

Fig. 5. Average Turnaround Time for Different Schedulers

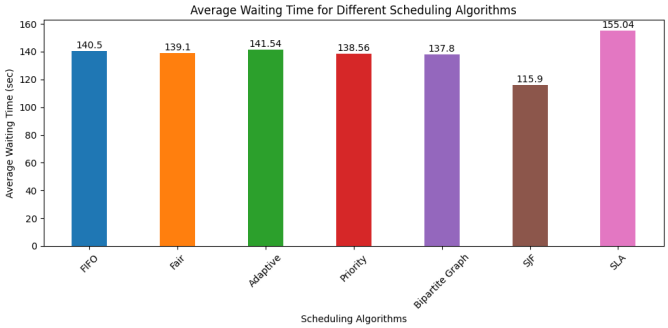

Fig. 6. Average Waiting Time for Different Schedulers

When we look at various job scheduling strategies, we see many interesting study opportunities. One direction to work on is to devise a hybrid scheduling approach formed by consolidating multiple algorithms. This integration has the potential of creating customized solutions, specialized in dealing with uniquely challenging datasets.

Another way can be utilizing machine learning and deep learning techniques to predict the workload. So, through predictive models, big data framework users can very actively control schedules based on resource-predicted needs to operate more responsive and efficient systems.

In addition, researchers can further develop and establish approaches to evaluate scheduling strategies more effectively with standard methods in order to have more reliable comparisons between them. It establishes guidelines to allow researchers a means for facilitating the comparison of the variations of different incurred processes for creating areas needed for improvement.

VI. Conclusion

Effective job scheduling is one of the most demanding aspects of big data processing. There are merits and demerits of scheduling algorithms based on Efficiency, Non-violation of SLA, average waiting time average turnaround time, etc. This research presents a comprehensive analysis of various schedulers, including FIFO (First In, First Out), Fair, Adaptive, Priority, Bipartite Graph, SJF (Shortest Job First), and SLA

schedulers. The research findings underscore the importance of selecting the appropriate scheduling algorithm based on specific workload characteristics and performance objectives, ensuring optimal resource utilization and system efficiency in big data processing environments. Researchers and practitioners need to consider various aspects of schedulers, and they can select the scheduler according to their workload. The result is effective for analyzing scheduling algorithms in modern data-driven environments. As a part of our future work, we like to perform an extensive and comprehensive experiment on a large heterogeneous real Apache Spark cluster to evaluate scheduler performance.

REFERENCES

[1] Jia D, Wang L, Valencia N, Bhimani J, Sheng B, Mi N. Learning-Based Dynamic Memory Allocation Schemes for Apache Spark Data Processing. IEEE Transactions on Cloud Computing. 2023 Nov 10.

[2] Lim S, Park D. Improving Hadoop MapReduce performance on heterogeneous single board computer clusters. Future Generation Computer Systems. 2024 Jun 15.

[3] Sai AA, Sahil G, Nadh BS, Eswar KL, Nisha KS, Prakash KR, Mahesh AD. Friend Recommendation System Using Map-Reduce and Spark: A Comparison Study. In 2023 4th International Conference on Innovative Trends in Information Technology (ICITIIT) 2023 Feb 11 (pp. 1-6). IEEE.

[4] Deepthi BG, Rani KS, Krishna PV, Saritha V. An efficient architecture for processing real-time traffic data streams using apache flink. Multimedia Tools and Applications. 2024 Apr;83(13):37369-85.

[5] Rodrigues M, Santos MY, Bernardino J. Big data processing tools: An experimental performance evaluation. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery. 2019 Mar;9(2):e1297.

[6] Raptis TP, Cicconetti C, Passarella A. Efficient topic partitioning of Apache Kafka for high-reliability real-time data streaming applications. Future Generation Computer Systems. 2024 May 1;154:173-88.

[7] Nasiri H, Nasehi S, Divband A, Goudarzi M. A scheduling algorithm to maximize storm throughput in heterogeneous cluster. Journal of Big Data. 2023 Jun 17;10(1):103.

[8] García-Gil D, Ramírez-Gallego S, García S, Herrera F. A comparison on scalability for batch big data processing on Apache Spark and Apache Flink. Big Data Analytics. 2017 Dec;2:1-1.

[9] Zhou R, Pang J, Zhang Q, Wu C, Jiao L, Zhong Y, Li Z. Online scheduling algorithm for heterogeneous distributed machine learning jobs. IEEE Transactions on Cloud Computing. 2022 Jan 14;11(2):1514-29.

[10] Fu Z, He M, Yi Y, Tang Z. Improving Data Locality of Tasks by Executor Allocation in Spark Computing Environment. IEEE Transactions on Cloud Computing. 2024 May 28.

[11] Sun X, He Y, Wu D, Huang JZ. Survey of distributed computing frameworks for supporting big data analysis. Big Data Mining and Analytics. 2023 Jan 26;6(2):154-69.

[12] Xu G, Xu CZ. MEER: Online estimation of optimal memory reservations for long lived containers in in-memory cluster computing. In2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS) 2019 Jul 7 (pp. 23-34). IEEE.

[13] Fan J, Han F, Liu H. Challenges of big data analysis. National science review. 2014 Jun 1;1(2):293-314.

[14] Zhang M, Lam KT, Yao X, Wang CL. Simpo: A scalable in-memory persistent object framework using nvram for reliable big data computing. ACM Transactions on Architecture and Code Optimization (TACO). 2018 Mar 22;15(1):1-28.

[15] Pan D, Yang Y. FIFO-based multicast scheduling algorithm for virtual output queued packet switches. IEEE Transactions on Computers. 2005 Aug 29;54(10):1283-97.

[16] Ciaburro G, Ayyadevara VK, Perrier A. Hands-on machine learning on google cloud platform: Implementing smart and efficient analytics using cloud ml engine. Packt Publishing Ltd; 2018 Apr 30.

[17] Kartik S, Murthy CS. Task allocation algorithms for maximizing reliability of distributed computing systems. IEEE Transactions on computers. 1997 Jun;46(6):719-24.

[18] Luu H, Luu H. Resilient Distributed Datasets. Beginning Apache Spark 2: With Resilient Distributed Datasets, Spark SQL, Structured Streaming and Spark Machine Learning library. 2018:51-86.

[19] Baresi L, Leva A, Quattrocchi G. Fine-grained dynamic resource allocation for big-data applications. IEEE Transactions on Software Engineering. 2019 Jul 29;47(8):1668-82.

[20] Kumar P, Rathore VS. Improvising and optimizing resource utilization in big data processing. InProceedings of Fifth International Conference on Soft Computing for Problem Solving: SocProS 2015, Volume 1 2016 (pp. 345-353). Springer Singapore.

[21] Zhang H, Cho B, Seyfe E, Ching A, Freedman MJ. Riffle: Optimized shuffle service for large-scale data analytics. InProceedings of the Thirteenth EuroSys Conference 2018 Apr 23 (pp. 1-15).

[22] Ruiu P, Scionti A, Nider J, Rapoport M. Workload management for power efficiency in heterogeneous data centers. In2016 10th International Conference on Complex, Intelligent, and Software Intensive Systems (CISIS) 2016 Jul 6 (pp. 23-30). IEEE.

[23] Jayanthi M, Rao KR. Experimental Setup of Apache Spark Application Execution in a Standalone Cluster Environment using Default Scheduling Mode. In2022 International Conference on Automation, Computing and Renewable Systems (ICACRS) 2022 Dec 13 (pp. 984-988). IEEE.

[24] Jayanthi M, Rao KR. Experimental Setup of Apache Spark Application Execution in a Standalone Cluster Environment using Default Scheduling Mode. In2022 International Conference on Automation, Computing and Renewable Systems (ICACRS) 2022 Dec 13 (pp. 984-988). IEEE.

[25] Gui Y, Tang D, Zhu H, Zhang Y, Zhang Z. Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. Computers & Industrial Engineering. 2023 Jun 1;180:109255.

[26] Mahes R, Mandjes M, Boon M, Taylor P. Adaptive scheduling in service systems: A Dynamic programming approach. European Journal of Operational Research. 2024 Jan 16;312(2):605-26.

[27] Islam MT, Karunasekera S, Buyya R. Performance and cost-efficient spark job scheduling based on deep reinforcement learning in cloud computing environments. IEEE Transactions on Parallel and Distributed Systems. 2021 Nov 2;33(7):1695-710.

[28] Yuan H, Bi J, Zhou M, Liu Q, Ammari AC. Biobjective task scheduling for distributed green data centers. IEEE Transactions on Automation Science and Engineering. 2020 Jan 7;18(2):731-42.

[29] Hu Z, Tu J, Li B. Spear: Optimized dependency-aware task scheduling with deep reinforcement learning. In2019 IEEE 39th international conference on distributed computing systems (ICDCS) 2019 Jul 7 (pp. 2037-2046). IEEE.

[30] Z. Hu and D. Li, "Improved heuristic job scheduling method to enhance throughput for big data analytics," in Tsinghua Science and Technology, vol. 27, no. 2, pp. 344-357, April 2022.

[31] Islam MT, Wu H, Karunasekera S, Buyya R. SLA-based scheduling of spark jobs in hybrid cloud computing environments. IEEE Transactions on Computers. 2021 Apr 27;71(5):1117-32.

[32] Cheng D, Chen Y, Zhou X, Gmach D, Milojicic D. Adaptive scheduling of parallel jobs in spark streaming. In IEEE INFOCOM 2017-IEEE Conference on Computer Communications 2017 May 1 (pp. 1-9). IEEE.

[33] Ajila T, Majumdar S. Data driven priority scheduling on spark based stream processing. In2018 IEEE/ACM 5th International Conference on Big Data Computing Applications and Technologies (BDCAT) 2018 Dec 17 (pp. 208-210). IEEE.

[34] Fu Z, Tang Z, Yang L, Liu C. An optimal locality-aware task scheduling algorithm based on bipartite graph modelling for spark applications. IEEE Transactions on Parallel and Distributed Systems. 2020 May 4;31(10):2406-20.

[35] Lu S, Zhao M, Li C, Du Q, Luo Y. Time-Aware Data Partition Optimization and Heterogeneous Task Scheduling Strategies in Spark Clusters. The Computer Journal. 2024 Feb;67(2):762-76.